Recursive Function to Convert a Decimal Number into its Binary Representation

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a recursive function to convert a decimal number into its binary representation.

2. Objective

- To understand the concept of recursion and how a function can call itself.
- To understand the decimal and binary number systems.
- To implement the repeated-division method for decimal-to-binary conversion using a recursive function in C.
- To test the program using different inputs, including edge cases.

3. Introduction

Decimal number system. The decimal system is a base-10 system that uses the digits 0 to 9. Each position in a number represents a power of 10.

Binary number system. The binary system is a base-2 system that uses only two digits, 0 and 1. Each position represents a power of 2. Computers store and process all data in binary, so converting between decimal and binary is a fundamental operation in computer science.

Recursion. Recursion is a technique in which a function calls itself to solve a smaller version of the same problem. Every recursive function needs a base case, which stops the recursion, and a recursive case, which reduces the problem towards the base case.

4. Method Used

The conversion uses the repeated division by 2 method. The decimal number is divided by 2 again and again, and the remainder of each division is noted. The binary number is obtained by reading the remainders in reverse order, from the last remainder to the first.

For example, to convert 10 into binary:

Division	Quotient	Remainder	Order of reading
10 ÷ 2	5	0	4th (last digit)
5 ÷ 2	2	1	3rd
2 ÷ 2	1	0	2nd
1 ÷ 2	0	1	1st (first digit)

Reading the remainders from bottom to top gives 1010. Hence, $(10)_{10} = (1010)_2$.

Recursion suits this method naturally. The recursive call is made first, which handles the higher-order digits, and the remainder is printed after the call returns. This automatically prints the remainders in reverse order, so no array or string reversal is needed.

5. Algorithm

decimalToBinary(n):

1. If n is equal to 0, return (base case).
2. Call decimalToBinary(n / 2) (recursive case).
3. After the call returns, print n % 2.

main():

1. Read the decimal number num from the user.
2. If num is negative, display an error message.
3. If num is 0, display 0 as the binary representation.
4. Otherwise, call decimalToBinary(num) and display the result.

6. Dry Run (n = 10)

The table below shows how the function executes for n = 10. The calls are made from top to bottom, and the digits are printed while the calls return from bottom to top.

Call	Value of n	Action	Printed on return
1	10	calls decimalToBinary(5)	10 % 2 = 0
2	5	calls decimalToBinary(2)	5 % 2 = 1
3	2	calls decimalToBinary(1)	2 % 2 = 0
4	1	calls decimalToBinary(0)	1 % 2 = 1
5	0	base case, returns	nothing

The calls return in the order 5, 4, 3, 2, 1, so the printed digits are 1, 0, 1, 0, giving the output 1010.

7. Source Code

```
#include <stdio.h>

/* Recursive function: prints the binary form of a positive integer n */
void decimalToBinary(int n)
{
    if (n == 0)                 /* base case: nothing left to divide */
        return;

    decimalToBinary(n / 2);     /* recursive call on the quotient    */
    printf("%d", n % 2);        /* print remainder while returning   */
}

int main()
{
    int num;

    printf("Enter a decimal number: ");
    scanf("%d", &num);

    if (num < 0)
    {
        printf("Please enter a non-negative number.\n");
    }
    else if (num == 0)
    {
        printf("Binary representation of 0 is: 0\n");
    }
    else
    {
        printf("Binary representation of %d is: ", num);
        decimalToBinary(num);
        printf("\n");
    }

    return 0;
}
```

8. Explanation of the Code

- Base case: when n becomes 0, the function returns without doing anything. This stops the recursion.
- Recursive case: the function calls itself with n / 2, which moves the number closer to 0.
- Printing: the statement printf("%d", n % 2) is placed after the recursive call. The remainders are therefore printed in reverse order of their calculation, which is exactly the binary form of the number.

- Special cases: 0 and negative numbers are handled in main() because the recursive function prints nothing for n = 0.

9. Sample Output

The program was compiled with gcc and tested with the following inputs.

Test 1: Input 10

```
Enter a decimal number: 10
Binary representation of 10 is: 1010
```

Test 2: Input 25

```
Enter a decimal number: 25
Binary representation of 25 is: 11001
```

Test 3: Input 255

```
Enter a decimal number: 255
Binary representation of 255 is: 11111111
```

Test 4: Input 0 (edge case)

```
Enter a decimal number: 0
Binary representation of 0 is: 0
```

Test 5: Input -5 (invalid input)

```
Enter a decimal number: -5
Please enter a non-negative number.
```

10. Complexity Analysis

Measure	Complexity	Reason
Time complexity	O(log n)	n is halved in every call, so about $\log_2(n) + 1$ calls are made.
Space complexity	O(log n)	Each recursive call occupies one stack frame, and the depth is about $\log_2(n)$.

11. Conclusion

The program successfully converts a decimal number into its binary representation using a recursive function. The repeated division method, combined with printing the remainder after the recursive call, produces the digits in the correct order without any extra data structure. The program works correctly for positive numbers and also handles the special cases of zero and negative input. This project helped in understanding recursion, the base case, the call stack, and number system conversion.